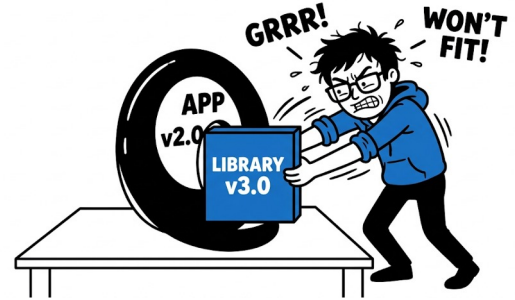


Practical Troubleshooting in *AI-Assisted* Development

1 Dependency Hell & Version Mismatch

The Issue: Installation failures (**Peer Dependency Conflict**) or runtime errors (**Module Not Found**) caused by AI assuming a generic environment while the local setup has version conflicts.

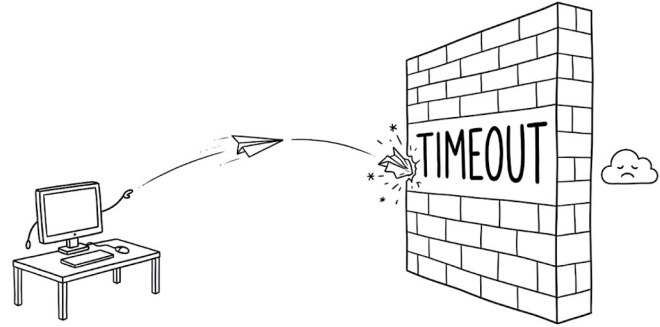


Agent Instruction (SOP):

- **Audit Runtime:** Always verify `node -v` or `python --version` before suggesting installs.
- **The "Clean Slate" Rule:** If installation fails, provide commands to delete lock files/folders (`node_modules`, `package-lock.json`) and retry.
- **Legacy Support:** Use compatibility flags like `npm install --legacy-peer-deps` or specific version pinning.

2 Network Hurdles & Proxy Walls

The Issue: Timeouts or **Connection Reset** errors during dependency downloads or API calls, often due to regional network restrictions or misconfigured proxies.

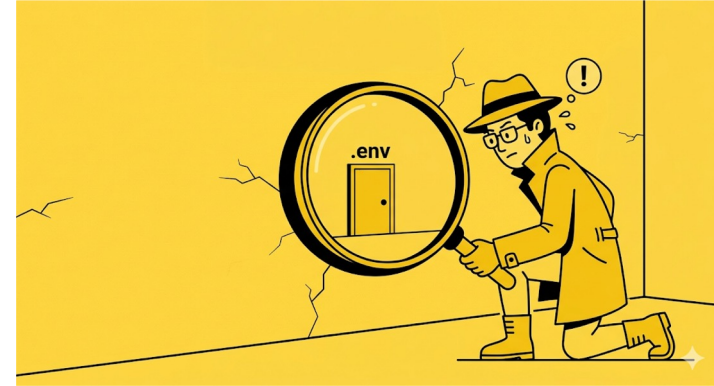


Agent Instruction (SOP):

- **Mirror Detection:** If a timeout occurs, do NOT change the code. Instead, suggest switching to a local mirror (e.g., npmmirror.com or [pypi.tuna](https://pypi.tuna.tsinghua.edu.cn)).
- **Proxy Check:** Instruct the user to verify terminal-specific environment variables like [http_proxy](#) and [https_proxy](#).
- **No-Retry Policy:** Stop trying the same URL after two failures; suggest a network diagnostic command instead.

3 The Invisible .env & Config Gaps

The Issue: MVP crashes with `process.env.KEY is undefined`.
The logic exists, but the physical `.env` file or API keys are missing or ignored by `.gitignore`.



Agent Instruction (SOP):

- **Mandatory Templates:** Whenever a new secret is introduced, immediately generate an `.env.example` file.
- **Pre-flight Verification:** Before running the app, check if the required keys exist in the environment.
- **The "Hidden File" Reminder:** Remind the user that files starting with a dot might be hidden in their file explorer.

4 Port Conflicts & Ghost Processes

The Issue: The dreaded `EADDRINUSE: port 3000` error caused by an old version of the app still running in the background or another service occupying the slot.



Agent Instruction (SOP):

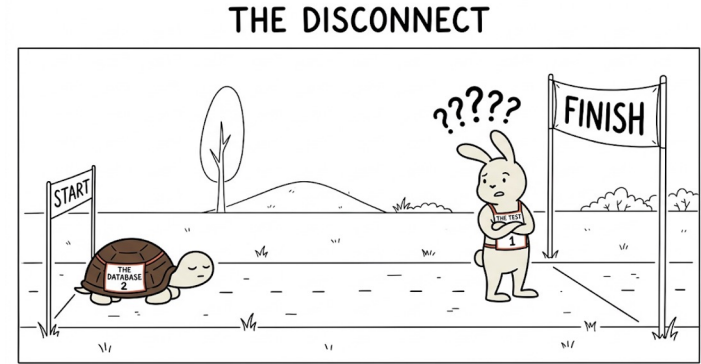
- **Port Pre-check:** Before suggesting a "start" command, run `lsof -i :[PORT]` to see if it's busy.
- **The Kill Command:** Proactively offer a "Search and Destroy" command (e.g., `fuser -k 3000/tcp`) to clear the port.
- **Dynamic Porting:** If the port is stuck, suggest incrementing the port number (e.g., 3000 to 3001) in the config.

5 Flaky Tests & Race Conditions

The Issue: Integration tests fail randomly (**Connection Refused** or **Element Not Found**) because the test starts before the database or server is fully "awake."

Agent Instruction (SOP):

- **Ban "Hard Sleeps":** Never use `sleep(5000)`. Instead, implement "Wait-for-Expect" or polling logic.
- **Service Warming:** Ensure the test script includes a "Health Check" ping to the API/DB before the first test case runs.
- **Clean Room Policy:** Add a "Teardown" step to wipe test data between runs to prevent "Dirty Data" errors.



6 Path Chaos & Context Loss

The Issue: **File not found** errors occur because the Agent assumes the root directory, while the user is executing from a sub-folder (or vice versa).



Agent Instruction (SOP):

- **Context Check:** Always run `pwd` before suggesting file-path-heavy commands.
- **Chain Commands:** Use `cd [target_dir] && [command]` to ensure the execution context is always correct.
- **Relative Paths:** Prefer relative paths based on the project root over absolute paths to ensure portability.

The "Environment-First" Golden Rule

The Principle: When a Terminal or integration test fails, assume the logic is innocent until the environment is proven guilty. Do not let the Agent rewrite code before verifying the physical execution layer.

(You can set this as Global rule for your AI coder)

The Troubleshooting Hierarchy (SOP for Agent): Follow this strict order of operations before any Prompt iteration:

1. **Path:** Is the Agent in the correct directory? (`pwd`)
2. **Dependency:** Are all packages installed and versions matched?
3. **Port:** Is the required port open and available?
4. **Environment:** Are the `.env` keys loaded and valid?
5. **Network:** Is there a proxy or connection blockage? *Only after these 5 checks pass should the Agent be allowed to modify the source code.*

